

Proyecto Programado II: Resolución de Laberintos

Instituto Tecnológico de Costa Rica

Sede Interuniversitaria de Alajuela

Santiago Escamilla Sanabria

29/05/2026

Resumen Ejecutivo

Este proyecto es un programa en Python que resuelve laberintos dados como matrices de texto. La idea es encontrar un camino desde el punto de inicio hasta el punto final usando backtracking con una pila, retrocediendo cada vez que se llega a un callejón sin salida.

El programa lee el laberinto desde la entrada, encuentra el camino (si existe) y lo marca. Si no hay ningún camino posible simplemente dice que no existe solución. Lo probé con 5 casos distintos y todos funcionaron..

Introducción

Resolver un laberinto es básicamente encontrar una ruta entre dos puntos evitando obstáculos, usando búsqueda exhaustiva y backtracking para llegar a la meta (objetivo).

El objetivo de este proyecto es implementar ese algoritmo en Python, recibiendo el laberinto como una matriz de caracteres y devolviendo el camino encontrado marcado con asteriscos, o indicando que no existe solución si es el caso.

Marco Teórico

Matrices

Una matriz es una estructura de datos organizada en filas y columnas. En este proyecto el laberinto es una lista de listas en Python, donde cada celda tiene un caracter que indica si es espacio libre, pared, inicio o fin.

Pilas

Una pila es una estructura en el que el último en entrar es el primero en salir. En Python se puede simular con una lista normal usando append para agregar elementos y pop para sacarlos. Aquí la use para guardar las celdas que faltan por explorar junto con el camino recorrido hasta ese punto.

Backtracking

El backtracking es una tecnica que consiste en explorar todas las posibilidades de forma ordenada, y cuando un camino no lleva a ningún lado simplemente se abandona y se vuelve atrás. En el caso del laberinto ese retroceso ocurre de forma automática cuando se hace pop de la pila, ya que eso nos devuelve a la celda anterior que todavía tenía opciones sin explorar.

Búsqueda Exhaustiva

La búsqueda exhaustiva recorre todos los caminos posibles hasta encontrar la solución o que no existe ninguna. Usando una la matriz de visitados, intenta que ninguna celda se procese muchas veces lo que da una complejidad de $O(F \times C)$ donde F es el número de filas y C el de columnas.

Descripción de la Solución

El programa lo dividí en funciones pequeñas donde cada una hace una cosa. Hay funciones para leer el laberinto, buscar el inicio, verificar si una celda es valida, resolver el laberinto con backtracking, marcar el camino encontrado y imprimirlo. La función main es la principal.

El algoritmo principal crea una matriz de visitados para no repetir celdas, luego mete el punto de inicio en la pila y empieza a explorar. En cada iteración saca la celda del tope de la pila, revisa si es el destino, y si no, agrega a la pila todos los vecinos validos con el camino actualizado. Si la pila se vacia sin haber encontrado el destino, significa que no hay solución.

Listing 1: Función principal de backtracking

```
1
2 def resolver(mapa, filas, cols, inicio):
3     # backtracking con pila y los 4 movimientos posibles
4     # si llega a donde no pueda seguir, retrocede automaticamente al hacer
        pop
5     dirs = [(-1,0), (1,0), (0,-1), (0,1)] # arriba, abajo, izq, der
6     visitados = [[False] * cols for _ in range(filas)]
7
8     # cada elemento de la pil
9     pila = [(inicio[0], inicio[1], [inicio])]
10    visitados[inicio[0]][inicio[1]] = True
11
12    while pila:
13        f, c, camino = pila.pop() #si no hay saliday volver hasta atras
14
15        if mapa[f][c] == 'E':
```

```
16         return camino #para encontrar el fin
17
18     for df, dc in dirs:
19         nf, nc = f + df, c + dc
20         if es_valida(nf, nc, filas, cols, mapa, visitados):
21             visitados[nf][nc] = True
22             pila.append((nf, nc, camino + [(nf, nc)]))
23
24     return None # pila vacia por si no hay solucion
25 # Agradecimientos a un compa que me ayudo a hacer esto
```

Resultados de Pruebas

Prueba 1 — Camino directo

Entrada:

3 5

S...E

#####

.....

Salida :

S***E

#####

.....

El camino es directo en la primera fila y el algoritmo lo encontró sin necesidad de retroceder.

Prueba 2 — Sin solución

Entrada:

3 3

S.#

###

#.E

Salida:

No existe solución

Las paredes bloquean completamente el paso entre S y E, la pila se vacía sin encontrar el destino.

Prueba 3 — Camino con retrocesos

Entrada:

5 5

S.#..

.####

.....

####.

....E

Salida:

S.#..

*####

####*

....E

Acá el algoritmo tuvo que explorar hacia abajo y serpentear por el laberinto, demostrando el retroceso cuando intentaba caminos sin salida.

Prueba 4 — Laberinto grande

Entrada:

8 10

S.....

.#####.

.....

.#####.

.....

.#####.

.....

.....E

Salida:

S*****

.#####*

*#####.

.#####*

.....*

.....E

Sirve para verificar que el programa funciona bien con laberintos mas grandes y no se cae.

Prueba 5 — Múltiples caminos

Entrada:

4 6

S....E

.....

.....

.....

Salida:

S****E

.....

.....

.....

Hay muchos caminos posibles en este caso. El algoritmo elige el camino directo por la primera fila dependiendo del orden en que la pila explora las direcciones.

Conclusiones

- El backtracking con pila es una forma limpia de resolver este tipo de problemas porque el retroceso ocurre solo con el pop, sin nada más.
- La matriz de visitados es esencial para que el algoritmo no entre en ciclos y sea eficiente.
- El programa maneja bien todos los casos que se piden, con solución, sin solución, caminos largos, laberintos grandes y múltiples caminos posibles.
- La complejidad es $O(F \times C)$ en el peor caso ya que cada celda se visita a lo sumo una vez.

Aprendizajes

- Aprendí a implementar backtracking de forma iterativa con pila en vez de recursión.
- Me di cuenta que es mejor usar funciones aux.
- Empece a usar casos de prueba que cubran todos (o casi todos) los escenarios posibles.